

## CS23232-Fundamentals of Data Structures Using C-2025 Batch

|              |                                   |
|--------------|-----------------------------------|
| Started on   | Wednesday, 25 March 2026, 8:14 AM |
| State        | Finished                          |
| Completed on | Wednesday, 25 March 2026, 9:24 AM |
| Time taken   | 1 hour 10 mins                    |
| Name         | PAVITHRA B 2025-BME               |

## Question 1

Correct

Marked out of 1.00

Flag question

Write a program to find the minimum and second minimum elements with in the elements of one dimensional array.

## Constraints:

- $1 \leq N \leq 10^3$
- $1 \leq \text{Elements of the array} \leq 10^6$

**Instruction:** To run your custom test cases strictly map your input and output layout with the visible test cases.

## Hints

Let us assume that first element itself as the **minimum**, **second\_minimum** and then compare both with all the other elements.

If any one found as **minimum** then change the value of the minimum, otherwise compare it with **second\_minimum** and exchange it when necessary condition is satisfied.

## For example:

| Input            | Result                                      |
|------------------|---------------------------------------------|
| 4<br>65 32 85 96 | Min element = 32<br>Second min element = 65 |

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 #include<limits.h>
3 int main()
4 {
5     int n;
6     scanf("%d",&n);
7     int arr[n];
8     for(int i = 0; i < n; i++)
9     {
10         scanf("%d",&arr[i]);
11     }
12     int min = INT_MAX, second_min = INT_MAX;
13     for(int i = 0; i < n; i++)
14     {
15         if(arr[i] < min)
16         {
17             second_min = min;
18             min = arr[i];
19         }
20     }
21     else if(arr[i] < second_min && arr[i] != min)
22 
```

|   | Input            | Expected                                    | Got                                         |   |
|---|------------------|---------------------------------------------|---------------------------------------------|---|
| ✓ | 4<br>65 32 85 96 | Min element = 32<br>Second min element = 65 | Min element = 32<br>Second min element = 65 | ✓ |

Passed all tests! ✓

## Question 2

Correct

Marked out of 1.00

Flag question

Write a program to read a student n subjects marks in an array and find the total, average of the marks.

For example, if the user gives the **input** as:

3

Next, the program should print the messages one by one on the console.

If the user gives the **input** as:

75

80

85

then the program should **print** the result as:

The total marks = 240

The average marks = 80.000000

## Hints

**marks** are integers, **total** is also an integer but **average** is a float value, so typecast it.

|              |                                   |
|--------------|-----------------------------------|
| Started on   | Wednesday, 25 March 2026, 8:14 AM |
| State        | Finished                          |
| Completed on | Wednesday, 25 March 2026, 9:24 AM |
| Time taken   | 1 hour 10 mins                    |
| Name         | PAVITHRA B 2025-BME               |

## Question 1

Correct

Marked out of 1.00

Flag question

Write a program to find the minimum and second minimum elements with in the elements of one dimensional array.

**Constraints:**

- $1 \leq N \leq 10^3$
- $1 \leq \text{Elements of the array} \leq 10^6$

**Instruction:** To run your custom test cases strictly map your input and output layout with the visible test cases.

**Hints**

Let us assume that first element it self as the **minimum**, **second\_minimum** and then compare both with all the other elements.

If any one found as **minimum** then change the value of the minimum, otherwise compare it with **second\_minimum** and exchange it when necessary condition is satisfied.

**For example:**

| Input       | Result                  |
|-------------|-------------------------|
| 4           | Min element = 32        |
| 65 32 85 96 | Second min element = 65 |

**Answer:** (penalty regime: 0 %)

```

1 #include<stdio.h>
2 #include<limits.h>
3 int main()
4 {
5     int n;
6     scanf("%d",&n);
7     int arr[n];
8     for(int i = 0; i < n; i++)
9     {
10         scanf("%d",&arr[i]);
11     }
12     int min = INT_MAX, second_min = INT_MAX;
13     for(int i = 0; i < n; i++)
14     {
15         if(arr[i] < min)
16         {
17             second_min = min;
18             min = arr[i];
19         }
20
21         else if(arr[i] < second_min && arr[i] != min)
22     }
```

|   | Input       | Expected                | Got                     |   |
|---|-------------|-------------------------|-------------------------|---|
| ✓ | 4           | Min element = 32        | Min element = 32        | ✓ |
|   | 65 32 85 96 | Second min element = 65 | Second min element = 65 |   |

Passed all tests! ✓

## Question 2

Correct

Marked out of 1.00

Flag question

Write a program to read a student n subjects marks in an array and find the total, average of the marks.

For example, if the user gives the **input** as:

3

Next, the program should print the messages one by one on the console.

if the user gives the **input** as:

75

80

85

then the program should **print** the result as:

The total marks = 240

The average marks = 80.000000

**Hints**

**marks** are integers, **total** is also an integer but **average** is a float value, so typecast it.

For example, if the user gives the **input** as:

3

Next, the program should print the messages one by one on the console.

if the user gives the **input** as:

75

80

85

then the program should **print** the result as:

The total marks = 240

The average marks = 80.000000

Hints

**marks** are integers, **total** is also an integer but **average** is a float value, so typecast it.

For example:

| Input | Result                        |
|-------|-------------------------------|
| 5     | The total marks = 325         |
| 45    | The average marks = 65.000000 |
| 65    |                               |
| 55    |                               |
| 75    |                               |
| 85    |                               |
| 4     | The total marks = 175         |
| 36    | The average marks = 43.750000 |
| 45    |                               |
| 38    |                               |
| 56    |                               |

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6     int marks[n];
7     int total = 0;
8     for(int i = 0; i < n; i++)
9     {
10         scanf("%d", &marks[i]);
11         total += marks[i];
12     }
13     float average = (float)total / n;
14     printf("The total marks = %d\n", total);
15     printf("The average marks = %.6f", average);
16     return 0;
17 }
```

|                     | Input | Expected                      | Got                           |   |
|---------------------|-------|-------------------------------|-------------------------------|---|
| ✓                   | 5     | The total marks = 325         | The total marks = 325         | ✓ |
|                     | 45    | The average marks = 65.000000 | The average marks = 65.000000 |   |
|                     | 65    |                               |                               |   |
|                     | 55    |                               |                               |   |
|                     | 75    |                               |                               |   |
|                     | 85    |                               |                               |   |
| ✓                   | 4     | The total marks = 175         | The total marks = 175         | ✓ |
|                     | 36    | The average marks = 43.750000 | The average marks = 43.750000 |   |
|                     | 45    |                               |                               |   |
|                     | 38    |                               |                               |   |
|                     | 56    |                               |                               |   |
| Passed all tests! ✓ |       |                               |                               |   |

The below sample code finds the addition of two matrices.

In the main() function read a two two dimensional array of elements and then find the addition of two matrices.

The logic is

- First checks the **row sizes** of two dimensional arrays are equal or not.
- If the sizes are not equal then, **stop the process**.
- If the sizes are equal then use **two for loops** to add each corresponding elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired output.



Saved

Marked out of  
1.00

Flag question

For example, if the user gives the **input** as:

3

Next, the program should print the messages one by one on the console.

if the user gives the **input** as:

75

80

85

then the program should **print** the result as:

The total marks = 240

The average marks = 80.000000

Hints

**marks** are integers, **total** is also an integer but **average** is a float value, so typecast it.

For example:

| Input | Result                        |
|-------|-------------------------------|
| 5     | The total marks = 325         |
| 45    | The average marks = 65.000000 |
| 65    |                               |
| 55    |                               |
| 75    |                               |
| 85    |                               |
| 4     | The total marks = 175         |
| 36    | The average marks = 43.750000 |
| 45    |                               |
| 38    |                               |
| 56    |                               |

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6     int marks[n];
7     int total = 0;
8     for(int i = 0; i<n; i++)
9     {
10         scanf("%d", &marks[i]);
11         total += marks[i];
12     }
13     float average = (float)total / n;
14     printf("The total marks = %d\n", total);
15     printf("The average marks = %.6f", average);
16     return 0;
17 }
```

|   | Input | Expected                      | Got                           |   |
|---|-------|-------------------------------|-------------------------------|---|
| ✓ | 5     | The total marks = 325         | The total marks = 325         | ✓ |
|   | 45    | The average marks = 65.000000 | The average marks = 65.000000 |   |
|   | 65    |                               |                               |   |
|   | 55    |                               |                               |   |
|   | 75    |                               |                               |   |
|   | 85    |                               |                               |   |
| ✓ | 4     | The total marks = 175         | The total marks = 175         | ✓ |
|   | 36    | The average marks = 43.750000 | The average marks = 43.750000 |   |
|   | 45    |                               |                               |   |
|   | 38    |                               |                               |   |
|   | 56    |                               |                               |   |

Passed all tests! ✓

Passed all tests! ✓

The below sample code finds the addition of two matrices.

In the main() function read a two two-dimensional array of elements and then find the addition of two matrices.

The logic is

- First checks the **row sizes** and **column sizes** of two two-dimensional arrays are equal or not.
- If the sizes are not equal then print "Addition is not possible" and stop the process.
- If the sizes are equal then use **two for loops** to add each corresponding elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired output.

For example:

| Input   | Result                      |
|---------|-----------------------------|
| 2 2     | The given matrix-1 is       |
| 1 2 3 4 | 1 2                         |
| 2 2     | 3 4                         |
| 4 5 6 7 | The given matrix-2 is       |
|         | 4 5                         |
|         | 6 7                         |
|         | Addition of two matrices is |
|         | 5 7                         |
|         | 9 11                        |

Answer: (penalty regime: 0 %)

Reset answer

```
3 = {
4     int i, j, m, n, p, q;
5     int a[5][5], b[5][5], c[5][5];
6     scanf("%d %d", &m, &n);
7     for (i = 0; i < m; i++)
8     { // Complete the code in for
9         for (j = 0; j < n; j++)
10        { // Complete the code in for
11            scanf("%d", &a[i][j]);
12        }
13    }
14    scanf("%d %d", &p, &q);
15    for (i = 0; i < p; i++)
16    { // Complete the code in for
17        for (j = 0; j < q; j++)
18        { // Complete the code in for
19            scanf("%d", &b[i][j]);
20        }
21    }
22    printf("The given matrix-1 is\n");
23    for(i = 0; i < m; i++)
24    {
```

|   | Input   | Expected                    | Got                         |   |
|---|---------|-----------------------------|-----------------------------|---|
| ✓ | 2 2     | The given matrix-1 is       | The given matrix-1 is       | ✓ |
|   | 1 2 3 4 | 1 2                         | 1 2                         |   |
|   | 2 2     | 3 4                         | 3 4                         |   |
|   | 4 5 6 7 | The given matrix-2 is       | The given matrix-2 is       |   |
|   |         | 4 5                         | 4 5                         |   |
|   |         | 6 7                         | 6 7                         |   |
|   |         | Addition of two matrices is | Addition of two matrices is |   |
|   |         | 5 7                         | 5 7                         |   |
|   |         | 9 11                        | 9 11                        |   |

Passed all tests! ✓

The below sample code finds the multiplication of two matrices.

In the main() function read a two two-dimensional array of elements and then find the multiplication of two matrices.

The logic is

- First check the **column size** of **first matrix** and **row size** of **second matrix** are equal or not.
- If the sizes are not equal then print "multiplication is not possible" and stop the process.
- If the sizes are equal then use **three for loops** to multiply the elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired result.

For example:

| Input | Result                |
|-------|-----------------------|
| 2 3   | The given matrix-1 is |



Passed all tests! ✓

Question 3

Correct

Marked out of 1.00

Flag question

The below sample code finds the addition of two matrices.

In the main() function read a two two-dimensional array of elements and then find the addition of two matrices.

The logic is

- First checks the **row sizes** and **column sizes** of two two-dimensional arrays are equal or not.
- If the sizes are not equal then print 'Addition is not possible' and stop the process.
- If the sizes are equal then use **two for loops** to add each corresponding elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired output.

For example:

| Input   | Result                      |
|---------|-----------------------------|
| 2 2     | The given matrix-1 is       |
| 1 2 3 4 | 1 2                         |
| 2 2     | 3 4                         |
| 4 5 6 7 | The given matrix-2 is       |
|         | 4 5                         |
|         | 6 7                         |
|         | Addition of two matrices is |
|         | 5 7                         |
|         | 9 11                        |

Answer: (penalty regime: 0 %)

Reset answer

```

3 {
4     int i, j, m, n, p, q;
5     int a[5][5], b[5][5], c[5][5];
6     scanf("%d %d", &m, &n);
7     for (i = 0; i < m; i++)
8     { // Complete the code in for
9         for (j = 0; j < n; j++)
10        { // Complete the code in for
11            scanf("%d", &a[i][j]);
12        }
13    }
14    scanf("%d %d", &p, &q);
15    for (i = 0; i < p; i++)
16    { // Complete the code in for
17        for (j = 0; j < q; j++)
18        { // Complete the code in for
19            scanf("%d", &b[i][j]);
20        }
21    }
22    printf("The given matrix-1 is\n");
23    for(i = 0; i < m; i++)
24    {

```

|   | Input   | Expected                    | Got                         |   |
|---|---------|-----------------------------|-----------------------------|---|
| ✓ | 2 2     | The given matrix-1 is       | The given matrix-1 is       | ✓ |
|   | 1 2 3 4 | 1 2                         | 1 2                         |   |
|   | 2 2     | 3 4                         | 3 4                         |   |
|   | 4 5 6 7 | The given matrix-2 is       | The given matrix-2 is       |   |
|   |         | 4 5                         | 4 5                         |   |
|   |         | 6 7                         | 6 7                         |   |
|   |         | Addition of two matrices is | Addition of two matrices is |   |
|   |         | 5 7                         | 5 7                         |   |
|   |         | 9 11                        | 9 11                        |   |

Passed all tests! ✓

Question 4

Correct

Marked out of 1.00

Flag question

The below sample code finds the multiplication of two matrices.

In the main() function read a two two-dimensional array of elements and then find the multiplication of two matrices.

The logic is

- First check the **column size of first matrix** and **row size of second matrix** are equal or not.
- If the sizes are not equal then print "multiplication is not possible" and stop the process.
- If the sizes are equal then use **three for loops** to multiply the elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired result.

For example:

| Input | Result                |
|-------|-----------------------|
| 2 3   | The given matrix-1 is |

Passed all tests! ✓

The below sample code finds the multiplication of two matrices.

In the main() function read a two two-dimensional array of elements and then find the multiplication of two matrices.

The logic is

- First check the **column size of first matrix** and **row size of second matrix** are equal or not.
- If the sizes are not equal then print 'multiplication is not possible' and stop the process.
- If the sizes are equal then use **three for loops** to multiply the elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired result.

For example:

| Input       | Result                            |
|-------------|-----------------------------------|
| 2 3         | The given matrix-1 is             |
| 1 2 3 5 7 9 | 1 2 3                             |
| 3 2         | 5 7 9                             |
| 3 2 1 5 6 7 | The given matrix-2 is             |
|             | 3 2                               |
|             | 1 5                               |
|             | 6 7                               |
|             | Multiplication of two matrices is |
|             | 23 33                             |
|             | 76 108                            |

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, j, k, m, n, p, q;
5     int a[5][5], b[5][5], c[5][5];
6     scanf("%d %d", &m, &n);
7     for (i = 0; i < m; i++)
8     { // Complete the code in for
9         for (j = 0; j < n; j++)
10        { // Complete the code in for
11            scanf("%d", &a[i][j]);
12        }
13    }
14    scanf("%d %d", &p, &q);
15    for (i = 0; i < p; i++)
16    { // Complete the code in for
17        for (j = 0; j < q; j++)
18        { // Complete the code in for
19            scanf("%d", &b[i][j]);
20        }
21    }
22    printf("The given matrix-1 is\n");
```

|   | Input                                    | Expected                                                                                                                                      | Got                                                                                                                                           |   |
|---|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---|
| ✓ | 2 3<br>1 2 3 5 7 9<br>3 2<br>3 2 1 5 6 7 | The given matrix-1 is<br>1 2 3<br>5 7 9<br>The given matrix-2 is<br>3 2<br>1 5<br>6 7<br>Multiplication of two matrices is<br>23 33<br>76 108 | The given matrix-1 is<br>1 2 3<br>5 7 9<br>The given matrix-2 is<br>3 2<br>1 5<br>6 7<br>Multiplication of two matrices is<br>23 33<br>76 108 | ✓ |

Passed all tests! ✓

Write a program to find the largest and second largest elements with in the elements of the given one dimensional array.

For example, if the user gives the **input** as:

5

Next, the program should print the messages one by one on the console.

if the user gives the **input** as:

10  
50  
30  
20  
25

then the program should **print** the result as:

The largest element of the array = 50

Passed all tests! ✓

Question 4

Correct

Marked out of 1.00

Flag question

The below sample code finds the multiplication of two matrices.

In the main() function read a two two-dimensional array of elements and then find the multiplication of two matrices.

The logic is

- First check the **column size of first matrix** and **row size of second matrix** are equal or not.
- If the sizes are not equal then print "multiplication is not possible" and stop the process.
- If the sizes are equal then use **three for loops** to multiply the elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired result.

For example:

| Input                                    | Result                                                                                                                                        |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 2 3<br>1 2 3 5 7 9<br>3 2<br>3 2 1 5 6 7 | The given matrix-1 is<br>1 2 3<br>5 7 9<br>The given matrix-2 is<br>3 2<br>1 5<br>6 7<br>Multiplication of two matrices is<br>23 33<br>76 108 |

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, j, k, m, n, p, q;
5     int a[5][5], b[5][5], c[5][5];
6     scanf("%d %d", &m, &n);
7     for (i = 0; i < m; i++)
8     { // Complete the code in for
9         for (j = 0; j < n; j++)
10        { // Complete the code in for
11            scanf("%d", &a[i][j]);
12        }
13    }
14    scanf("%d %d", &p, &q);
15    for (i = 0; i < p; i++)
16    { // Complete the code in for
17        for (j = 0; j < q; j++)
18        { // Complete the code in for
19            scanf("%d", &b[i][j]);
20        }
21    }
22    printf("The given matrix-1 is\n");
```

|   | Input                                    | Expected                                                                                                                                      | Got                                                                                                                                           |   |
|---|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---|
| ✓ | 2 3<br>1 2 3 5 7 9<br>3 2<br>3 2 1 5 6 7 | The given matrix-1 is<br>1 2 3<br>5 7 9<br>The given matrix-2 is<br>3 2<br>1 5<br>6 7<br>Multiplication of two matrices is<br>23 33<br>76 108 | The given matrix-1 is<br>1 2 3<br>5 7 9<br>The given matrix-2 is<br>3 2<br>1 5<br>6 7<br>Multiplication of two matrices is<br>23 33<br>76 108 | ✓ |

Passed all tests! ✓

Question 5

Correct

Marked out of 1.00

Flag question

Write a program to find the largest and second largest elements with in the elements of the given one dimensional array.

For example, if the user gives the input as:

5

Next, the program should print the messages one by one on the console.

if the user gives the input as:

10  
50  
30  
20  
25

then the program should print the result as:

The largest element of the array = 50  
The second largest element of the array = 30



10  
50  
30  
20  
25

then the program should **print** the result as:

The largest element of the array = 50  
The second largest element of the array = 30

#### Hints

Let us assume first element it self as the **large**, **second\_large** and then compare both with all the other elements.  
If any one found as **large** then change the value of the large, otherwise compare it with **second\_large** and exchange it when necessary condition is satisfied.

For example:

| Input                                 | Result                                                                                |
|---------------------------------------|---------------------------------------------------------------------------------------|
| 5<br>2<br>5<br>9<br>7<br>3            | The largest element of the array = 9<br>The second largest element of the array = 7   |
| 6<br>64<br>87<br>34<br>58<br>62<br>18 | The largest element of the array = 87<br>The second largest element of the array = 64 |

Answer: (penalty regime: 0%)

```
1 #include<stdio.h>
2 #include<limits.h>
3 int main()
4 {
5     int n,i;
6     int arr[100];
7     int largest = INT_MIN, secondlargest = INT_MIN;
8     if(scanf("%d",&n)!= 1 || n <= 0)
9     {
10         printf("Invalid input for number of elements.\n");
11         return 1;
12     }
13     for(i=0;i<n;i++)
14     {
15         if(scanf("%d",&arr[i])!= 1)
16         {
17             printf("Invalid input for array element.\n");
18             return 1;
19         }
20     }
21     for(i=0;i<n;i++)
22     {
```

|                     | Input                                 | Expected                                                                              | Got                                                                            |
|---------------------|---------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| ✓                   | 5<br>2<br>5<br>9<br>7<br>3            | The largest element of the array = 9<br>The second largest element of the array = 7   | The largest element of the array = 9<br>The second largest element of the arr  |
| ✓                   | 6<br>64<br>87<br>34<br>58<br>62<br>18 | The largest element of the array = 87<br>The second largest element of the array = 64 | The largest element of the array = 87<br>The second largest element of the arr |
| Passed all tests! ✓ |                                       |                                                                                       |                                                                                |

Finish review



Saved

10  
50  
30  
20  
25

then the program should **print** the result as:

The largest element of the array = 50  
The second largest element of the array = 30

#### Hints

Let us assume first element itself as the **large**, **second\_large** and then compare both with all the other elements. If any one found as **large** then change the value of the large, otherwise compare it with **second\_large** and exchange it when necessary condition is satisfied.

For example:

| Input                                 | Result                                                                                |
|---------------------------------------|---------------------------------------------------------------------------------------|
| 5<br>2<br>5<br>9<br>7<br>3            | The largest element of the array = 9<br>The second largest element of the array = 7   |
| 6<br>64<br>87<br>34<br>58<br>62<br>18 | The largest element of the array = 87<br>The second largest element of the array = 64 |

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 #include<limits.h>
3 int main()
4 {
5     int n,i;
6     int arr[100];
7     int largest = INT_MIN, secondlargest = INT_MIN;
8     if(scanf("%d",&n)!= 1 || n <= 0)
9     {
10         printf("Invalid input for number of elements.\n");
11         return 1;
12     }
13     for(i=0;i<n;i++)
14     {
15         if(scanf("%d",&arr[i])!= 1)
16         {
17             printf("Invalid input for array element.\n");
18             return 1;
19         }
20     }
21     for(i=0;i<n;i++)
22     {
```

|   | Input                                 | Expected                                                                              | Got                                                                            |
|---|---------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| ✓ | 5<br>2<br>5<br>9<br>7<br>3            | The largest element of the array = 9<br>The second largest element of the array = 7   | The largest element of the array = 9<br>The second largest element of the arr  |
| ✓ | 6<br>64<br>87<br>34<br>58<br>62<br>18 | The largest element of the array = 87<br>The second largest element of the array = 64 | The largest element of the array = 87<br>The second largest element of the arr |

Passed all tests! ✓

The function `strcat()` is used to concatenate two strings into a single string.

The syntax of `strcat()` is `strcat(string1, string2);`.

where `string1`, `string2` are two different strings. Here `string2` is concatenated with `string1`, and the **concatenated string** is stored in `string1`.

In `strcat()` operation, **NULL character** (`'\0'`) of `string1` is **overwritten** by first character of `string2` and **NULL character** (`'\0'`) is appended (added) at the end of **new** `string1` which is created after `strcat()` operation.

Fill the missing code in the below program to display the **concatenated** string using `strcat()` function.

For example:

| Input          | Result     |
|----------------|------------|
| REC<br>Chennai | RECChennai |

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char str1[20], str2[20];
7     scanf("%s", str1);
8     scanf("%s", str2);
9     strcat(str1, str2);
10    //Concat str2 with str1
11    printf("%s\n", str1 ); // Correct the code
12    return 0;
13 }
```

|   | Input          | Expected   | Got        |   |
|---|----------------|------------|------------|---|
| ✓ | REC<br>Chennai | RECChennai | RECChennai | ✓ |

Passed all tests! ✓

The function `strcmp()` is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of `strcmp()` is `variable_name = strcmp (string1, string2);`.

Where `string1`, `string2` are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function `strcmp()` returns zero, both strings are **equal**.

If the function `strcmp()` returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)

If the function `strcmp()` returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)

Fill the missing code in the below program to compare two strings using `strcmp()` function.

For example:

| Input                        | Result                                                         |
|------------------------------|----------------------------------------------------------------|
| NarendraModi<br>narendramodi | The string narendramodi is higher than the string NarendraModi |
| Krishna<br>Godavari          | The string Krishna is higher than the string Godavari          |

# Fundamentals of Data Structures Using C-2025 Batch

n Sunday, 12 April 2026, 11:19 AM  
e Finished  
n Sunday, 12 April 2026, 11:30 AM  
n 10 mins 41 secs  
e PAVITHRA B 2025-BME

The function `strcat()` is used to concatenate two strings into a single string.

The syntax of `strcat()` is `strcat(string1, string2);`.

where `string1`, `string2` are two different strings. Here `string2` is concatenated with `string1`, and the **concatenated string** is stored in `string1`.

In `strcat()` operation, **NULL character** (`'\0'`) of `string1` is **overwritten** by first character of `string2` and **NULL character** (`'\0'`) is appended (added) at the end of **new string1** which is created after `strcat()` operation.

Fill the missing code in the below program to display the **concatenated** string using `strcat()` function.

For example:

| Input          | Result     |
|----------------|------------|
| REC<br>Chennai | RECChennai |

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char str1[20], str2[20];
7     scanf("%s", str1);
8     scanf("%s", str2);
9     strcat(str1, str2);
10    //Concat str2 with str1
11    printf("%s\n", str1); // Correct the code
12    return 0;
13 }
```

|   | Input          | Expected   | Got        |   |
|---|----------------|------------|------------|---|
| ✓ | REC<br>Chennai | RECChennai | RECChennai | ✓ |

Passed all tests! ✓

The function `strcmp()` is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of `strcmp()` is `variable_name = strcmp (string1, string2);`.

Where `string1`, `string2` are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function `strcmp()` returns zero, both strings are **equal**.

If the function `strcmp()` returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)

If the function `strcmp()` returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)

Fill the missing code in the below program to compare two strings using `strcmp()` function.



|              |                                 |
|--------------|---------------------------------|
| Started on   | Sunday, 12 April 2026, 11:19 AM |
| State        | Finished                        |
| Completed on | Sunday, 12 April 2026, 11:30 AM |
| Time taken   | 10 mins 41 secs                 |
| Name         | PAVITHRA B 2025-BME             |

## Question 1

Correct

Marked out of 1.00

Flag question

The function `strcat()` is used to concatenate two strings into a single string.

The syntax of `strcat()` is `strcat(string1, string2);`.

where `string1`, `string2` are two different strings. Here `string2` is concatenated with `string1`, and the **concatenated string** is stored in `string1`.

In `strcat()` operation, **NULL character** (`'\0'`) of `string1` is **overwritten** by first character of `string2` and **NULL character** (`'\0'`) is appended (added) at the end of **new** `string1` which is created after `strcat()` operation.

Fill the missing code in the below program to display the **concatenated** string using `strcat()` function.

For example:

| Input          | Result     |
|----------------|------------|
| REC<br>Chennai | RECChennai |

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char str1[20], str2[20];
7     scanf("%s", str1);
8     scanf("%s", str2);
9     strcat(str1, str2);
10    //Concat str2 with str1
11    printf("%s\n", str1); // Correct the code
12    return 0;
13 }
```

|   | Input          | Expected   | Got        |   |
|---|----------------|------------|------------|---|
| ✓ | REC<br>Chennai | RECChennai | RECChennai | ✓ |

Passed all tests! ✓

## Question 2

Correct

Marked out of 1.00

Flag question

The function `strcmp()` is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of `strcmp()` is `variable_name = strcmp (string1, string2);`.

Where `string1`, `string2` are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function `strcmp()` returns zero, both strings are **equal**.

If the function `strcmp()` returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)

If the function `strcmp()` returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)

Fill the missing code in the below program to compare two strings using `strcmp()` function.



Chennai

Passed all tests! ✓

The function `strcmp()` is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of `strcmp()` is `variable_name = strcmp (string1, string2);`.

Where `string1`, `string2` are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function `strcmp()` returns zero, both strings are **equal**.

If the function `strcmp()` returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)

If the function `strcmp()` returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)

Fill the missing code in the below program to compare two strings using **`strcmp()`** function.

For example:

| Input                        | Result                                                         |
|------------------------------|----------------------------------------------------------------|
| NarendraModi<br>narendramodi | The string narendramodi is higher than the string NarendraModi |
| Krishna<br>Godavari          | The string Krishna is higher than the string Godavari          |
| REC<br>REC                   | The given two strings are equal                                |

Answer: (penalty regime: 0 %)

Reset answer

```
2  #include <string.h>
3
4  int main()
5  {
6      char a[20], b[20];
7      int res;
8      scanf("%s", a);
9      scanf("%s", b);
10     res=strcmp(a,b);
11     //Compare two strings
12
13     if (res==0 )
14     { // Correct the code
15         printf("The given two strings are equal\n");
16     }
17     else if (res>0 )
18     { // Correct the code
19         printf("The string %s is higher than the string %s\n", a, b);
20     }
21     else
22     {
23         printf("The string %s is higher than the string %s\n", b, a);
```

|   | Input                        | Expected                                                       | Got          |
|---|------------------------------|----------------------------------------------------------------|--------------|
| ✓ | NarendraModi<br>narendramodi | The string narendramodi is higher than the string NarendraModi | The string r |
| ✓ | Krishna<br>Godavari          | The string Krishna is higher than the string Godavari          | The string l |
| ✓ | REC<br>REC                   | The given two strings are equal                                | The given tv |

Passed all tests! ✓

The function `strcpy()` is used to **copy** one string into another string including the NULL character (terminator `char '\0'`).

The syntax of `strcpy()` is `strcpy(string1, string2);`.

Where `string1`, `string2` are two strings and the `string2` is copied into `string1`. In this case the copied string is available in `string1` and both strings contains the same data.

If the length of `string1` is less than the length of `string2` then entire `string2` value will not be copied into `string1`.

For example, consider the length of `string1` is **20** and the length of `string2` is **30**. Then, only the first **20** characters from `string2` will be copied into `string1`, the remaining **10** characters will not be copied and will be **truncated**.

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

The function `strcmp()` is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of `strcmp()` is `variable_name = strcmp (string1, string2);`.

Where `string1`, `string2` are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function `strcmp()` returns zero, both strings are **equal**.

If the function `strcmp()` returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)

If the function `strcmp()` returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)

Fill the missing code in the below program to compare two strings using **`strcmp()`** function.

For example:

| Input                        | Result                                                         |
|------------------------------|----------------------------------------------------------------|
| NarendraModi<br>narendramodi | The string narendramodi is higher than the string NarendraModi |
| Krishna<br>Godavari          | The string Krishna is higher than the string Godavari          |
| REC<br>REC                   | The given two strings are equal                                |

Answer: (penalty regime: 0 %)

Reset answer

```
2 #include <string.h>
3
4 int main()
5 {
6     char a[20], b[20];
7     int res;
8     scanf("%s", a);
9     scanf("%s", b);
10    res=strcmp(a,b);
11    //Compare two strings
12
13    if (res==0 )
14    { // Correct the code
15        printf("The given two strings are equal\n");
16    }
17    else if (res>0 )
18    { // Correct the code
19        printf("The string %s is higher than the string %s\n", a, b);
20    }
21    else
22    {
23        printf("The string %s is higher than the string %s\n" b, a);
```

|   | Input                        | Expected                                                       | Got          |
|---|------------------------------|----------------------------------------------------------------|--------------|
| ✓ | NarendraModi<br>narendramodi | The string narendramodi is higher than the string NarendraModi | The string r |
| ✓ | Krishna<br>Godavari          | The string Krishna is higher than the string Godavari          | The string k |
| ✓ | REC<br>REC                   | The given two strings are equal                                | The given tw |

Passed all tests! ✓

Question 3

Correct

Marked out of 1.00

Flag question

The function `strcpy()` is used to **copy** one string into another string including the NULL character (terminator `char '\0'`).

The syntax of `strcpy( )` is `strcpy(string1, string2);`.

Where `string1`, `string2` are two strings and the `string2` is copied into `string1`. In this case the copied string is available in `string1` and both strings contains the same data.

If the length of `string1` is less than the length of `string2` then entire `string2` value will not be copied into `string1`.

For example, consider the length of `string1` is **20** and the length of `string2` is **30**. Then, only the first **20** characters from `string2` will be copied into `string1`, the remaining **10** characters will not be copied and will be **truncated**.

available in string1 and both strings contains the same data.

If the length of string1 is less than the length of string2 then entire string2 value will not be copied into string1.

For example, consider the length of string1 is **20** and the length of string2 is **30**. Then, only the first **20** characters from string2 will be copied into string1, the remaining **10** characters will not be copied and will be **truncated**.

Understand and retype the below code which demonstrates the usage of **strcpy()** function.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[20], str2[20];
    scanf("%s", str2);
    strcpy(str1, str2);
    printf("The copied string = %s", str1);
    return 0;
}
```

For example:

| Input | Result                   |
|-------|--------------------------|
| Rose  | The copied string = Rose |

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4
5     char str1[20],str2[20];
6     scanf("%s",str2);
7     strcpy(str1,str2);
8     printf("The copied string = %s\n",str1);
9     return 0;
10 }
```

|   | Input | Expected                 | Got                      |   |
|---|-------|--------------------------|--------------------------|---|
| ✓ | Rose  | The copied string = Rose | The copied string = Rose | ✓ |

Passed all tests! ✓

In C language, we have four types of string functions that are used for performing **string operations**. They are strlen(), strcpy(), strcat(), strcmp().

The function strlen() is used to find the **length** of the given string. This function returns only the **integer data** (or) **numeric data**.

The function strlen() counts the number of characters in a given string and returns the Integer value.

It stops counting the character when **NULL** character is found. Because, **NULL** character indicates the end of the string in C.

The syntax of strlen() is integer\_variable = strlen(string);.

Here string is a group of characters, strlen() function finds the **length** of the string and the **integer** value will be stored in the integer\_variable.

The string.h header file supports all the string functions in C language.

Fill in the missing code in the below program to find the **length** of a string using **strlen()** function.

For example:

| Input        | Result                                      |
|--------------|---------------------------------------------|
| NarendraModi | The length of the string NarendraModi is 12 |

Answer: (penalty regime: 0 %)

Reset answer

available in string1 and both strings contains the same data.

If the length of string1 is less than the length of string2 then entire string2 value will not be copied into string1.

For example, consider the length of string1 is **20** and the length of string2 is **30**. Then, only the first **20** characters from string2 will be copied into string1, the remaining **10** characters will not be copied and will be **truncated**.

Understand and retype the below code which demonstrates the usage of **strcpy()** function.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[20], str2[20];
    scanf("%s", str2);
    strcpy(str1, str2);
    printf("The copied string = %s", str1);
    return 0;
}
```

For example:

| Input | Result                   |
|-------|--------------------------|
| Rose  | The copied string = Rose |

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4
5     char str1[20],str2[20];
6     scanf("%s",str2);
7     strcpy(str1,str2);
8     printf("The copied string = %s\n",str1);
9     return 0;
10 }
```

|   | Input | Expected                 | Got                      |   |
|---|-------|--------------------------|--------------------------|---|
| ✓ | Rose  | The copied string = Rose | The copied string = Rose | ✓ |

Passed all tests! ✓

#### Question 4

Correct

Marked out of 1.00

Flag question

In C language, we have four types of string functions that are used for performing **string operations**. They are **strlen()**, **strcpy()**, **strcat()**, **strcmp()**.

The function **strlen()** is used to find the **length** of the given string. This function returns only the **integer data** (or) **numeric data**.

The function **strlen()** counts the number of characters in a given string and returns the integer value.

It stops counting the character when **NULL** character is found. Because, **NULL** character indicates the end of the string in C.

The syntax of **strlen()** is **integer\_variable = strlen(string);**.

Here string is a group of characters, **strlen()** function finds the **length** of the string and the **integer** value will be stored in the **integer\_variable**.

The **string.h** header file supports all the string functions in C language.

Fill in the missing code in the below program to find the **length** of a string using **strlen()** function.

For example:

| Input        | Result                                      |
|--------------|---------------------------------------------|
| NarendraModi | The length of the string NarendraModi is 12 |

Answer: (penalty regime: 0 %)

Reset answer



|   | Input | Expected                 | Got                      |   |
|---|-------|--------------------------|--------------------------|---|
| ✓ | Rose  | The copied string = Rose | The copied string = Rose | ✓ |

Passed all tests! ✓

In C language, we have four types of string functions that are used for performing **string operations**. They are strlen(), strcpy(), strcat(), strcmp().

The function strlen() is used to find the **length** of the given string. This function returns only the **integer data** (or) **numeric data**.

The function strlen() counts the number of characters in a given string and returns the integer value.

It stops counting the character when **NULL** character is found. Because, **NULL** character indicates the end of the string in C.

The syntax of strlen() is integer\_variable = strlen(string);.

Here string is a group of characters, strlen() function finds the **length** of the string and the **integer** value will be stored in the integer\_variable.

The string.h header file supports all the string functions in C language.

Fill in the missing code in the below program to find the **length** of a string using **strlen()** function.

For example:

| Input        | Result                                      |
|--------------|---------------------------------------------|
| NarendraModi | The length of the string NarendraModi is 12 |

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char ch[20];
7     scanf("%s", ch);
8     printf("The length of the string %s is %ld\n", ch, strlen(ch)); //Correct
9     return 0;
10 }
```

|   | Input        | Expected                                    | Got                             |
|---|--------------|---------------------------------------------|---------------------------------|
| ✓ | NarendraModi | The length of the string NarendraModi is 12 | The length of the string Narend |

Passed all tests! ✓



Saved

Finish review